



Universidade do Minho

Escola de Engenharia

Licenciatura em Engenharia Informática

Unidade Curricular de Computação Gráfica

Ano Letivo de 2025/2026

SolariUM

Criação de um motor gráfico

Luís Ferreira
a98286

José Fernandes
a106937

Gabriel Dantas
a107291

Simão Oliveira
a107322

Março, 2026

Índice

1. Introdução	1
2. Generator	2
2.1. Anel (Ring)	2
2.2. Octaedro	2
2.3. Scatter (Dispersão de Instâncias)	3
3. Engine	5
3.1. Estrutura do XML e Grupos	5
3.2. Leitura e Estruturas de Dados	5
3.3. Transformações geométricas	6
3.3.1. Translações	6
3.3.2. Rotações	6
3.3.3. Escalas	7
3.3.4. Composição de transformações geométricas	7
3.4. Renderização Recursiva	8
3.5. Câmara	8
3.5.1. Projeção — Frustum e Planos Near/Far	9
3.5.2. Modo de Câmara Livre	9
3.6. Funcionalidades de Depuração e Interação	10
4. Modelo do Sistema Solar	12
4.1. Planetas e Sol	12
4.2. Luas dos Planetas	13
4.3. Anéis de Saturno	13
4.4. Cinturões de Asteroides	14
4.5. Modelo do Asteroide	14
4.6. Conversor OBJ para .3d	14
4.7. Geração dos Cinturões	14
4.8. Estrelas	15
4.9. Detritos e Cometas	15
4.10. Resultado	15
5. Conclusão	17
Bibliografia	19
Lista de Siglas e Acrónimos	20
Anexos	21
Anexo 1 Logo da Universidade do Minho	21

Lista de Figuras

Figura 1	Representação do anel.	2
Figura 2	Representação do octaedro.	3
Figura 3	Resultado 1 — câmara orbital, distância inicial.	16
Figura 4	Resultado 2 — câmara orbital, distância atrás das estrelas.	16
Figura 5	Resultado 3 — câmara orbital, próxima do Sol.	16
Figura 6	Resultado 4 — câmara orbital, aproximada com <i>wireframe</i> ativo.	16
Figura 7	Resultado 5 — câmara livre, anéis de Saturno.	16
Figura 8	Resultado 6 — câmara livre, cinturão de asteroides.	16
Figura 9	Resultado 7 — câmara livre, luas de Júpiter.	16
Figura 10	Resultado 8 — câmara livre, Plutão e cinturão de <i>Kuiper</i>	16

1. Introdução

O presente relatório pretende descrever o processo de desenvolvimento do trabalho laboratorial desenvolvido no âmbito da Unidade Curricular de Computação Gráfica.

Com base no enunciado fornecido o objetivo é criar um sistema completo de geração e visualização de cenas tridimensionais, composto por duas aplicações principais: o *generator*, responsável pela criação de modelos geométricos, e a *engine*, encarregue da leitura de ficheiros XML, carregamento de modelos e renderização das cenas utilizando a API *OpenGL* [1]. O trabalho é desenvolvido de forma incremental, estando dividido em quatro fases distintas, cada uma introduzindo novos conceitos fundamentais da Computação Gráfica.

Na 1ª fase, foi implementada a leitura do ficheiro XML no arranque da aplicação e criadas as estruturas de dados adequadas para armazenar a informação da cena em memória. O *generator* produz modelos geométricos básicos e a *engine* carregá-os e desenhá-os.

Durante a presente 2ª fase, deve ser implementado o suporte a transformações geométricas hierárquicas (translação, rotação e escala), organizando a cena sob a forma de uma árvore de grupos. É necessário aplicar corretamente as transformações aos modelos e subgrupos. O cenário demonstrativo pedido é um modelo estático do sistema solar.

2. Generator

No *generator* foram acrescentadas três novas primitivas, por virem a ser úteis na construção do modelo do sistema solar, pedido nesta fase.

2.1. Anel (Ring)

O anel é uma superfície plana anular definida por dois raios — o interior, seja r_i , e o exterior, seja r_e — e por um número de fatias, seja s , que controla a resolução angular. Geometricamente, trata-se de um disco com um orifício central, orientado no plano XZ com $y = 0$.

Para construir a malha, divide-se o ângulo $[0, 2\pi]$ em s intervalos iguais, obtendo o espaçamento angular, $\Delta\theta = \frac{2\pi}{s}$. Iterando sobre $i \in [0, s - 1]$, definem-se os ângulos $\theta_1 = i \cdot \Delta\theta$ e $\theta_2 = (i + 1) \cdot \Delta\theta$. Em cada iteração, os quatro vértices do *quad* correspondente são:

$$\begin{cases} p_1 = (r_i \cdot \cos(\theta_1), 0, r_i \cdot \sin(\theta_1)) \\ p_2 = (r_e \cdot \cos(\theta_1), 0, r_e \cdot \sin(\theta_1)) \\ p_3 = (r_i \cdot \cos(\theta_2), 0, r_i \cdot \sin(\theta_2)) \\ p_4 = (r_e \cdot \cos(\theta_2), 0, r_e \cdot \sin(\theta_2)) \end{cases}$$

Cada *quad* é subdividido em dois triângulos em ordem CCW, tal como definido na função auxiliar descrita na Secção 2.1 do primeiro relatório, resultando em $2 \times s$ triângulos no total. A Figura 1 esquematiza esta construção.

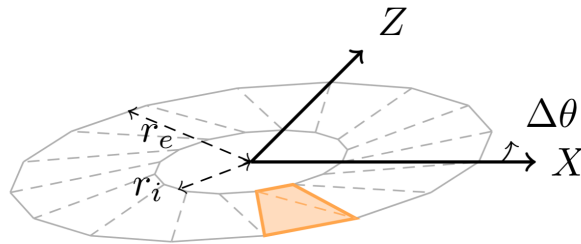


Figura 1: Representação do anel.

2.2. Octaedro

O octaedro é um poliedro convexo composto por 8 faces triangulares e 6 vértices, obtido pela interseção de dois quadrados rodados entre si. É utilizado para representar

estrelas no modelo do sistema solar, tirando partido da sua aparência aproximadamente esférica à distância independentemente do ângulo de câmara.

Para o construir, definem-se os 6 vértices canónicos dispostos nos semi-eixos positivo e negativo de X , Y e Z , escalados por um factor s e trasladados para a posição (x, y, z) [2]:

$$\begin{cases} v_0 = (s + x, 0 + y, 0 + z) \\ v_1 = (-s + x, 0 + y, 0 + z) \\ v_2 = (0 + x, s + y, 0 + z) \\ v_3 = (0 + x, -s + y, 0 + z) \\ v_4 = (0 + x, 0 + y, s + z) \\ v_5 = (0 + x, 0 + y, -s + z) \end{cases}$$

As 8 faces são geradas em dois grupos simétricos — a pirâmide superior (apoiada em v_4) e a pirâmide inferior (apoiada em v_5) — cada uma com 4 triângulos laterais:

Superior: $(v_4, v_2, v_0), (v_4, v_1, v_2), (v_4, v_3, v_1), (v_4, v_0, v_3)$

Inferior: $(v_5, v_2, v_0), (v_5, v_1, v_2), (v_5, v_3, v_1), (v_5, v_0, v_3)$

A ordem dos vértices em cada triângulo respeita a orientação CCW para garantir a correta visibilidade das faces. A Figura 2 representa esta construção.

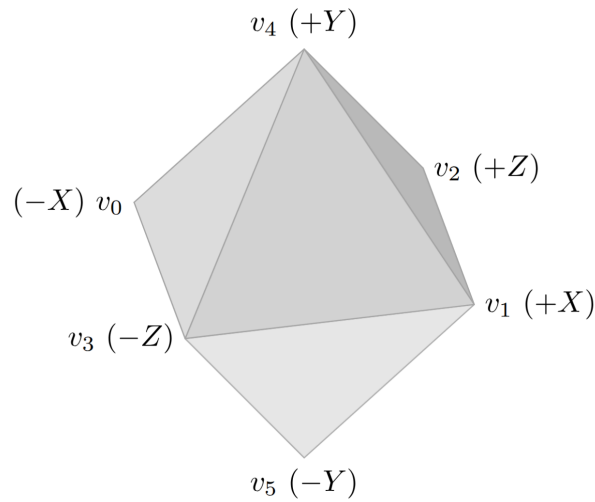


Figura 2: Representação do octaedro.

2.3. Scatter (Dispersão de Instâncias)

O *scatter* é um meta-gerador que combina dois conceitos: um volume de distribuição que define onde colocar objetos, e um modelo externo que define o que colocar. Permite popular qualquer região do espaço com N instâncias de um modelo .3d existente, cada uma com posição, rotação e escala aleatórias. A interface de linha de comandos é:

```
./generator scatter <volume> <params volume...> <N> <modelo.3d> <s_min>
<s_max> <saida.3d>
```

Em que $s_{\min}$ e $s_{\max}$ correspondem às limitações inferior e superior na escala aleatória destes modelos.

Os volumes de distribuição suportados e os respectivos parâmetros são os seguintes:

Volume	Parâmetros	Descrição
sphere	$r_{\min}$ $r_{\max}$	Casca esférica entre dois raios
torus	R $r_{\min}$ $r_{\max}$	Anel toroidal com raio de tubo variável
plane	width height	Superfície plana no plano XZ
cylinder	$r_{\min}$ $r_{\max}$ $h_{\min}$ $h_{\max}$	Casca cilíndrica com altura variável
box	inner_half outer_half	Casca cúbica entre dois tamanhos

Para cada instância, o *scatter*:

- Calcula uma posição aleatória dentro do volume com distribuição uniforme;
- Gera uma escala aleatória entre $s_{\min}$ e $s_{\max}$;
- Gera três ângulos de rotação aleatórios (r_x, r_y, r_z) em $[0, 2\pi]$;
- Aplica a transformação $scale \rightarrow rotateX \rightarrow rotateY \rightarrow rotateZ \rightarrow translate$ a cada vértice do modelo;
- Acumula os vértices transformados na lista de saída.

A transformação de cada vértice é realizada analiticamente, sem recurso a matrizes — a rotação é aplicada eixo a eixo por sequência de operações trigonométricas. O resultado é escrito diretamente no ficheiro `.3d` de saída, mantendo a semântica do formato: o modelo final é apenas uma lista de triângulos, indistinguível de qualquer outra primitiva. A *engine* não precisa de saber que o ficheiro foi gerado por *scatter*.

O *scatter* foi utilizado para gerar os dois cinturões de asteroides do modelo do sistema solar e o céu estrelado, o que iremos explorar mais em frente. Segue-se um exemplo de utilização que, dado um modelo de uma bola, cria dentro de uma casca cúbica um conjunto de 40 bolas:

```
./generator scatter box 0.5 5.0 40 ball.3d 0.3 1.0 balls_in_box.3d
```

3. Engine

Com as alterações ao *generator* explicadas, passamos agora ao que é novo no funcionamento da *engine*.

3.1. Estrutura do XML e Grupos

Para a aplicação desta etapa do projeto, o XML organiza-se de uma maneira diferente. Existem grupos, i.e., conjuntos de modelos, a que se podem aplicar transformações geométricas. Assim, num arquivo XML, é possível representar vários grupos diferentes. As transformações geométricas são executadas pela *engine* na ordem em que surgem. Veja-se o exemplo da Listagem 1.

```
<group>
  <transform>
    <rotate angle="280" x="0" y="1" z="0"/>
    <translate x="90" y="0" z="0"/>
    <scale x="0.53" y="0.53" z="0.53"/>
  </transform>
  <models>
    <model file="sphere.3d" color="#E27B58"/>
  </models>
</group>
```

Listagem 1: Exemplo de um grupo no XML.

3.2. Leitura e Estruturas de Dados

O ficheiro XML é lido na inicialização através da biblioteca TinyXML2 [3], que produz uma árvore de nós. A *engine* percorre essa árvore recursivamente para construir em memória uma hierarquia de grupos.

A estrutura central é o Group, que agrega três listas:

- *transforms* — lista ordenada de transformações geométricas a aplicar ao grupo;
- *models* — lista de modelos 3D (ficheiros .3d com cor associada) a desenhar neste grupo;
- *children* — lista de subgrupos, que herdam as transformações acumuladas do pai.

Cada transformação é representada por uma estrutura Transform com os campos: tipo (TRANSLATE, ROTATE ou SCALE), vetor (x, y, z) e ângulo α (usado apenas nas rota-

ções). A ordem de leitura das transformações a partir do XML é preservada na lista, pois a composição de transformações não é comutativa.

Os modelos são carregados do disco uma única vez: a *engine* lê o ficheiro .3d, interpreta a primeira linha como o número de vértices e as linhas seguintes como coordenadas (x, y, z) , armazenando-as num `vector<float>`. Cada modelo é identificado pelo nome do ficheiro, permitindo que a mesma geometria seja reutilizada por múltiplos grupos sem duplicação em memória.

3.3. Transformações geométricas

Durante esta fase foram implementadas, na *engine*, as transformações geométricas de translação, rotação e escala.

3.3.1. Translações

Uma translação desloca todos os pontos de um grupo segundo um vetor (t_x, t_y, t_z) , movendo-os no espaço tridimensional sem alterar a sua orientação ou escala. No XML, é declarada através do elemento `<translate>` com os atributos `x`, `y` e `z`, que por omissão assumem o valor 0:

```
<translate x="90" y="0" z="0"/>
```

Durante o carregamento do XML, o elemento é reconhecido pelo nome do nó e os seus atributos são lidos com a função `FloatAttribute`. Um objeto de transformação de tipo `TRANSLATE` é instanciado e adicionado à lista de transformações do grupo correspondente. Na fase de renderização, quando a *engine* processa a lista de transformações de um grupo, ao encontrar um elemento de tipo `TRANSLATE` invoca:

```
glTranslatef(t.x, t.y, t.z);
```

Esta chamada multiplica a matriz de *modelview* atual pela matriz de translação correspondente ao vetor (x, y, z) . Dado que as transformações são aplicadas dentro de um par `glPushMatrix / glPopMatrix`, o efeito da translação limita-se ao grupo atual e a todos os seus subgrupos, não se propagando para grupos irmãos ou para o grupo pai.

3.3.2. Rotações

Uma rotação gira os modelos de um grupo em torno de um eixo arbitrário definido pelo vetor (t_x, t_y, t_z) , segundo um ângulo, α , expresso em graus. No XML, é declarada pelo elemento `<rotate>` com os atributos *angle*, *x*, *y* e *z*:

```
<rotate angle="280" x="0" y="1" z="0"/>
```

Neste exemplo, a rotação é de 280° em torno do eixo Y. O atributo *angle* é lido e armazenado no campo homónimo da estrutura de transformação, enquanto o vetor (x, y, z) define o eixo de rotação. Na renderização, ao deparar-se com um elemento de tipo `ROTATE`, a *engine* invoca:

```
glRotatef(t.angle, t.x, t.y, t.z);
```

Internamente, o *OpenGL* constrói a matriz de rotação para o eixo e ângulo fornecidos e multiplica-a pela matriz de *modelview* corrente. O eixo não precisa de estar normalizado, pois o *OpenGL* trata dessa normalização automaticamente. Nesta fase, a rotação é puramente estática, i.e., o ângulo é fixo e lido diretamente do XML, não variando com o tempo.

3.3.3. Escalas

Uma escala redimensiona os modelos de um grupo ao longo dos três eixos de forma independente, multiplicando cada coordenada pelo fator correspondente. Quando os três fatores são iguais, obtém-se uma escala uniforme, o que iremos usar praticamente sempre, já quando diferem, o resultado é uma escala não-uniforme que deforma proporcionalmente o modelo nos eixos pretendidos. No XML, é declarada pelo elemento `<scale>` com os atributos *x*, *y* e *z*, com valor padrão 1:

```
<scale x="0.53" y="0.53" z="0.53"/>
```

No exemplo acima, todos os modelos do grupo ficam com 53% do seu tamanho original. O valor por omissão de cada componente é 1.0, assegurando que a ausência de um atributo não altera a geometria. Na renderização, ao encontrar um elemento de tipo *SCALE*, a *engine* invoca:

```
glScalef(t.x, t.y, t.z);
```

Esta chamada multiplica a matriz de *modelview* corrente pela matriz de escala diagonal com os fatores (s_x, s_y, s_z) . Tal como nas restantes transformações, o efeito está circunscrito ao grupo e aos seus subgrupos, sendo revertido automaticamente com o `glPopMatrix` no final da chamada recursiva.

3.3.4. Composição de transformações geométricas

A composição das transformações geométricas [4] é feita de forma multiplicativa e transparente, ou seja, executando as projeções através da multiplicação da matriz *modelview* corrente na ordem em que surgem no arquivo XML. Para cada grupo, a função de desenho faz a operação de *push* à matriz da projeção, altera-a, multiplicando a matriz da transformação e aplica recursivamente a mesma lógica aos subgrupos.

Sendo $R_{v,\alpha}$ a representação de uma matriz de rotação por α no eixo v , T_v a representação matricial de uma translação pelo vetor v e S_v a representação de uma matriz de escala por v , a projeção final, P , é dada por:

$$P = \prod_{i=1}^n M_i$$

Onde M_i representa a composição de transformações geométricas a um dado grupo:

$$M = f_1 \cdot f_2 \cdot \dots \cdot f_n,$$

$$f_i \in \{T_v, R_{v,\alpha}, S_v\}$$

3.4. Renderização Recursiva

A renderização da cena é feita por uma função recursiva `drawGroup` que recebe um grupo e aplica o seguinte algoritmo:

```
void drawGroup(const Group& g) {
    glPushMatrix();
    for (auto& t : g.transforms)
        applyTransform(t);
    for (auto& m : g.models)
        drawModel(m);
    for (auto& child : g.children)
        drawGroup(child);
    glPopMatrix();
}
```

Listagem 2: Função de renderização recursiva.

O par `glPushMatrix()` / `glPopMatrix()` garante que as transformações de um grupo não se propagam para os grupos irmãos nem para o grupo pai, confinando o efeito à subárvore. A função `applyTransform` despacha para `glTranslatef`, `glRotatef` ou `glScalef` conforme o tipo da transformação. A ordem de aplicação segue a ordem de declaração no XML, o que permite compor transformações arbitrárias de forma intuitiva. Esta separação de dados hierárquicos com um desenho recursivo usando uma pilha de matrizes mantém o código limpo e facilita animações e agrupamentos.

A função `drawModel` é responsável por enviar a geometria de um modelo à GPU. Para cada modelo, itera sobre os vértices previamente carregados em memória — armazenados num `vector<float>` em grupos de três coordenadas (x, y, z) — e envia-os ao *pipeline* gráfico através do mecanismo imediato do OpenGL.

Cada grupo de três vértices consecutivos define um triângulo, cuja orientação CCW é garantida pelo *generator*. A cor do modelo, lida a partir do atributo `color` do XML, é aplicada uma única vez antes do ciclo com `glColor3ubv`.

Note-se que esta abordagem com `glBegin` / `glEnd` envia os vértices à GPU individualmente em cada *frame*. Embora funcional para cenas de complexidade moderada, torna-se um *bottleneck* para cenas densas como os cinturões de asteroides gerados pelo *scatter*. A migração para *Vertex Buffer Objects* (VBOs), que transferem a geometria para memória da GPU uma única vez, está identificada como trabalho futuro na Secção 5.

Esta separação entre dados hierárquicos (`drawGroup`) e envio de geometria (`drawModel`) mantém o código modular e facilita a futura substituição do modo imediato por VBOs sem alterar a lógica de travessia da árvore de grupos.

3.5. Câmara

A câmara é lida a partir do bloco `<camera>` do XML e armazena três conjuntos de parâmetros: a posição (`position`), o ponto de interesse (`lookAt`), o vetor up e os parâmetros de projeção (`fov`, `near`, `far`).

3.5.1. Projeção — Frustum e Planos Near/Far

A projeção perspectiva é configurada através de `gluPerspective`, que constrói internamente uma matriz de projeção que define o volume de visualização — o *frustum*. O *frustum* é um tronco de pirâmide delimitado por seis planos: esquerdo, direito, superior, inferior, *near* e *far*.

O plano *near* define a distância mínima a que um objeto tem de estar da câmara para ser renderizado. O plano *far* define a distância máxima — objetos além deste plano são descartados. A escolha destes valores tem impacto direto na qualidade do *z-buffer*: o *z-buffer* tem precisão finita (tipicamente 24 bits) distribuída pelo intervalo $[near, far]$ de forma não-linear. Quanto maior o rácio $far / near$, menor a precisão disponível para distinguir objetos próximos, podendo causar *z-fighting* — artefactos visuais onde superfícies sobrepostas cintilam. Para o modelo do sistema solar, os valores foram calibrados para acomodar o raio da esfera de estrelas mantendo *near* suficientemente grande:

```
<projection fov="45" near="1" far="2000" />
```

Listagem 3: Configuração da projeção no XML.

O campo de visão (*fov*) determina o ângulo vertical da pirâmide de projeção. A relação de aspeto é calculada automaticamente a partir das dimensões da janela. A posição e orientação da câmara são aplicadas através de `gluLookAt`, que constrói a *view matrix* a partir dos parâmetros lidos do XML.

3.5.2. Modo de Câmara Livre

Para além da câmara fixa definida no XML, a *engine* disponibiliza um modo de câmara livre que permite ao utilizador navegar pela cena como se se movesse dentro dela, sem estar preso a um ponto de interesse fixo.

A câmara é definida pela sua posição P , um vetor de direção *forward* $\hat{f}$ e um vetor *up* $\hat{u}$. A cada *frame*, a *view matrix* é reconstruída a partir destes três vetores através de `gluLookAt()`.

O utilizador controla seis graus de liberdade:

- **Movimento para a frente/trás** (teclas w s) — translada a câmara ao longo de $\hat{f}$;
- **Movimento lateral** (teclas a d) — translada a câmara ao longo do vetor *right* $\hat{r} = \hat{f} \times \hat{u}$;
- **Rotação horizontal** (rato, eixo X) — roda $\hat{f}$ em torno de $\hat{u}$ (yaw);
- **Rotação vertical** (rato, eixo Y) — roda $\hat{f}$ em torno de $\hat{r}$ (pitch), com *clamping* para evitar a inversão nos polos.

A posição é atualizada a cada *frame* por:

$$P' = P + v_f \cdot \hat{f} + v_r \cdot \hat{r}$$

onde v_f e v_r é a velocidade de movimento, configurável através das teclas + e -. O vetor *forward* é recalculado após cada rotação, garantindo que o movimento é sempre relativo à orientação atual da câmara e não a um eixo global.

O estado da câmara é atualizado através de `glutPostRedisplay()` [5], que sinaliza o GLUT para redesenhar a cena apenas quando ocorrem alterações efetivas, evitando processamento redundante.

3.6. Funcionalidades de Depuração e Interação

A *engine* disponibiliza um conjunto de funcionalidades de depuração e interação acessíveis em tempo de execução, sem necessidade de recompilar ou reiniciar a aplicação. Estas funcionalidades são controladas através do teclado e resumidas num menu de opções acessível através da tecla M no terminal.

O modo *wireframe*, ativado com a tecla W, substitui o preenchimento sólido dos triângulos por apenas as suas arestas, permitindo inspecionar a malha geométrica de qualquer modelo da cena. É particularmente útil para verificar a densidade e distribuição dos triângulos gerados pelo *generator*.

O *back-face culling*, controlado pela tecla C, descarta automaticamente as faces traseiras dos modelos durante a renderização. Uma vez que o *generator* garante a orientação CCW de todos os triângulos, o *culling* pode ser ativado com segurança, reduzindo para metade o número de faces processadas pela GPU e melhorando a performance da cena.

Tecla	Ação
Opções de visualização	
W	Alternar modo <i>wireframe</i>
C	Alternar <i>back-face culling</i>
O	Mostrar/esconder contador de FPS
A	Mostrar/esconder eixos do referencial
Câmara orbital	
I / K	Rotação vertical
J / L	Rotação horizontal
+ / -	<i>Zoom in/out</i>
Rato (direito)	Rotação por arrasto
Câmara livre	
F	Alternar entre câmara orbital e livre
W / S	Mover para a frente/trás
A / D	Mover para a esquerda/direita
Rato	Orientar direção de visão (<i>yaw/pitch</i>)
Geral	
R	Recarregar configuração XML
M	Mostrar menu de opções
ESC	Sair

Tabela 1: Atalhos de teclado disponíveis na *engine*.

A visualização dos eixos, ativada com A, desenha os três eixos do referencial global diretamente na cena, servindo de referência espacial durante a exploração.

O contador de FPS, ativado com O, exibe em tempo real a cadência de fotogramas, permitindo avaliar o impacto de cada funcionalidade na performance.

Uma funcionalidade particularmente útil é o recarregamento dinâmico da configuração, acionado pela tecla R. Ao pressionar R, a *engine* relê o ficheiro XML e recarrega todos os modelos sem fechar a janela, refletindo imediatamente quaisquer alterações feitas ao ficheiro de cena. Isto permite iterar rapidamente sobre o posicionamento dos planetas, escalas e hierarquia de grupos sem interromper o fluxo de trabalho.

A Tabela 1 demonstra os atalhos que foram implementados para o programa de atalhos.

4. Modelo do Sistema Solar

O modelo do sistema solar é descrito num ficheiro XML que organiza todos os corpos celestes numa hierarquia de grupos com transformações geométricas. Cada planeta é posicionado através de uma rotação em torno do eixo Y seguida de uma translação ao longo do eixo X, o que equivale a colocá-lo numa posição orbital estática. Utilizou-se uma escala de conveniência visual baseada na Terra como unidade de referência. Este método mantém a ordem de magnitude relativa entre os planetas, garantindo que a proporção entre eles seja respeitada, apesar de não utilizar as distâncias e raios reais para evitar problemas de oclusão e visibilidade.

4.1. Planetas e Sol

O Sol, os oito planetas e Plutão são representados por icoesferas — esferas geradas por subdivisões de um icosaedro. A cada modelo é atribuída uma cor aproximada à aparência real do corpo e um tamanho em escala arbitrária escolhida para manter alguma proporcionalidade, mas para não deixar os tamanhos dos corpos celestes demasiado grandes ou pequenos em comparação com outras, dificultando uma visualização agradável:

Corpo	Escala	Cor (hexadecimal)	Cor (Aparência)
Sol	21.84	 #FDB813	Amarelo-alaranjado
Mercúrio	0.38	 #888888	Cinzentos
Vénus	0.95	 #E6E6FA	Branco-acinzentado
Terra	1.00	 #2B82C9	Azul
Marte	0.53	 #E27B58	Vermelho-alaranjado
Júpiter	11.20	 #C88B3A	Castanho-amarelado
Saturno	9.45	 #E3D599	Amarelo-pálido
Urano	4.00	 #4CB7D6	Azul-ciano
Neptuno	3.88	 #274687	Azul-escuro
Plutão	0.58	 #C2A98A	Castanho-claro

Tabela 2: Corpos celestes do modelo do sistema solar — escalas relativas à Terra e cores utilizadas na renderização.

A Lua da Terra é representada como um subgrupo do grupo da Terra, herdando assim a translação do planeta e sendo posicionada com uma translação adicional relativa. Este agrupamento hierárquico é um dos pontos-chave da estrutura de grupos implementada nesta fase.

4.2. Luas dos Planetas

As luas dos planetas foram representadas recorrendo ao mecanismo *scatter*, distribuindo icoesferas de menor detalhe (geradas com argumento de duas subdivisões) em cascas esféricas centradas em cada planeta. O número de instâncias geradas corresponde ao número real de luas conhecidas de cada planeta, e os raios da casca foram calibrados em função da escala do planeta no modelo — entre $1.5\times$ e $2\times$ o seu raio visual — de forma a que as luas fiquem visualmente associadas ao corpo que orbitam sem se sobrepor a ele nem ficar demasiado dispersas.

Os comandos utilizados para gerar os ficheiros de luas foram os seguintes:

```
./generator scatter sphere 14 20 95 icosphere_2.3d 0.04 0.30  
moons_jupiter.3d  
./generator scatter sphere 12 18 146 icosphere_2.3d 0.04 0.25  
moons_saturn.3d  
./generator scatter sphere 5 8 28 icosphere_2.3d 0.04 0.14  
moons_uranus.3d  
./generator scatter sphere 5 8 16 icosphere_2.3d 0.04 0.18  
moons_neptune.3d  
./generator scatter sphere 1 2 2 icosphere_2.3d 0.05 0.09  
moons_mars.3d
```

Listagem 4: Geração das luas dos planetas com o *scatter*.

Mercúrio e Vénus não têm luas e foram omitidos. A Terra mantém a Lua definida explicitamente no XML como subgrupo, conforme descrito na secção anterior. Plutão, classificado como planeta anão e situado dentro do cinturão de Kuiper, tem a sua lua Caronte também definida explicitamente dado o seu tamanho proporcionalmente grande — cerca de metade do diâmetro de Plutão.

Apesar de conceptualmente interessante, a representação das luas revelou-se visualmente confusa na prática: com 95 luas em torno de Júpiter e 146 em torno de Saturno, a nuvem de pontos dificulta a leitura da cena e retira clareza à estrutura do sistema solar. Por este motivo, foram mantidos dois ficheiros XML distintos — um com luas e outro sem — permitindo alterar entre as duas representações.

4.3. Anéis de Saturno

Os anéis de Saturno são simplificados e construídos com base em um anel gerado com a primitiva *ring*, os raios interior e exterior são calibrados em função do tamanho do planeta.

4.4. Cinturões de Asteroides

O sistema solar possui dois cinturões de asteroides conhecidos: o cinturão principal, situado entre Marte e Júpiter, e o cinturão de *Kuiper*, situado além de Neptuno. Ambos foram gerados com o mecanismo *scatter*, usando um volume de *torus* como região de distribuição.

4.5. Modelo do Asteroide

O modelo base do asteroide foi obtido a partir de um ficheiro *OBJ* disponível gratuitamente online [6], que representa uma forma rochosa irregular com algum detalhe. Uma vez que as coordenadas originais do modelo estavam em unidades arbitrárias com valores na ordem das centenas, foi desenvolvido um conversor em *Python* que, além de converter o formato *OBJ* para o formato *.3d* da *engine*, realiza uma normalização automática da geometria.

4.6. Conversor OBJ para .3d

O conversor *obj_to_3d.py* realiza três operações sequenciais sobre o modelo de entrada:

- Leitura dos vértices (linhas “v x y z”) e faces (linhas “f ...”) do ficheiro *OBJ*, com suporte a faces triangulares e quads (trianguladas em leque);
- Normalização: cálculo do centróide da nuvem de vértices, subtração do centróide a todos os vértices (centragem na origem) e divisão pelo maior *extent* absoluto (vértice mais afastado da origem), de forma a que o modelo caiba exatamente numa caixa de coordenadas ± 1 ;
- Escrita do ficheiro *.3d* com o número total de vértices expandidos na primeira linha, seguido das coordenadas de cada vértice dos triângulos, sem indexação.

A normalização é ativa por omissão e pode ser desativada com a opção `--no-normalize` para modelos que já se encontrem na escala correta. Após a conversão, o modelo do asteroide ocupa uma caixa ± 1 , pelo que os parâmetros de escala do *scatter* (*s_min*, *s_max*) passam a ter um significado intuitivo diretamente em unidades do sistema solar.

4.7. Geração dos Cinturões

O cinturão principal foi gerado distribuindo 200 instâncias do asteroide normalizado num volume de *torus* com raio central de 110 unidades (entre Marte em 90 e Júpiter em 130) e espessura de tubo entre 8 e 15 unidades. O cinturão de *Kuiper* foi gerado com 500 instâncias num *torus* de raio 290 unidades e espessura entre 12 e 25 unidades, refletindo a sua maior extensão e dispersão:

```
./generator scatter torus 110 8 15 200 asteroid.3d 0.1 0.5 belt_main.3d
./generator scatter torus 290 12 25 500 asteroid.3d 0.1 0.5
belt_kuiper.3d
```

Listagem 5: Geração dos cinturões de asteróides com o *scatter*.

Cada instância recebe uma rotação completamente aleatória nos três eixos, conferindo ao conjunto a aparência caótica e orgânica característica de um cinturão de asteroides real. O resultado de cada *scatter* é um único ficheiro .3d que a *engine* carrega e desenha de forma idêntica a qualquer outra primitiva, sem conhecimento de como o ficheiro foi produzido.

4.8. Estrelas

O fundo estrelado é também gerado pelo mecanismo *scatter*, distribuindo N octaedros de pequena escala sobre uma casca esférica de raio próximo do plano *far* da câmara, criando a ilusão de um céu estrelado fixo:

```
./generator scatter sphere 700 750 3000 octahedron.3d 0.3 0.8 stars.3d
```

A distribuição uniforme sobre a esfera é obtida por amostragem em coordenadas esféricas com correção angular, evitando a concentração de pontos nos polos:

```
float theta = 2.0f * M_PI * u;
float phi = acos(2.0f * v - 1.0f);
```

Onde u e v são amostras independentes com distribuição uniforme em $[0, 1]$.

Cada octaedro recebe ainda uma rotação aleatória nos três eixos, variando o aspeto visual de cada estrela. O resultado é um único ficheiro *stars.3d* carregado pela *engine* de forma idêntica a qualquer outra primitiva.

4.9. Detritos e Cometas

Por mero complemento, decidiram-se adicionar por toda a superfície esférica do sistema solar alguns detritos espaciais (o mesmo modelo dos asteroides) e numa área mais externa três objetos celestes maiores, semelhantes a cometas. Estes, geraram-se através do *scatter* com os comandos:

```
./generator scatter sphere 35 260 80 asteroid.3d 0.2 1.5 debris.3d
./generator scatter sphere 240 260 3 asteroid.3d 5 6 comets.3d
```

Listagem 6: Geração de detritos e cometas dispersos com o *scatter*.

4.10. Resultado

Seguem algumas imagens demonstrando o resultado adquirido.

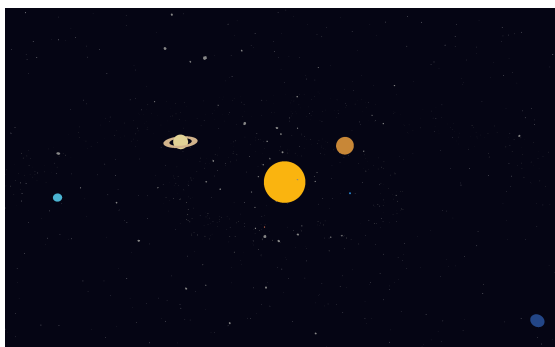


Figura 3: Resultado 1 — câmara orbital, distância inicial.

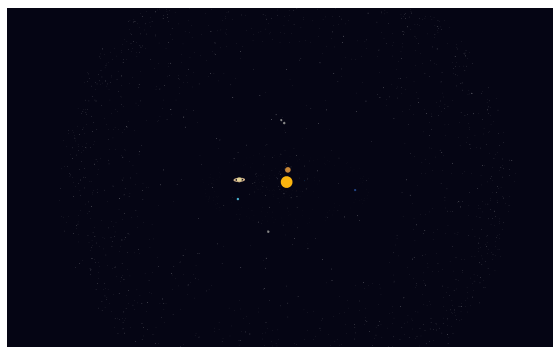


Figura 4: Resultado 2 — câmara orbital, distância atrás das estrelas.

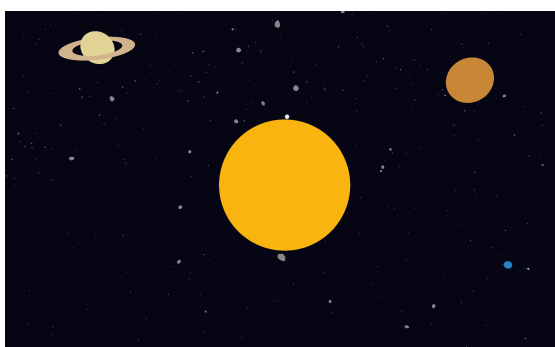


Figura 5: Resultado 3 — câmara orbital, próxima do Sol.

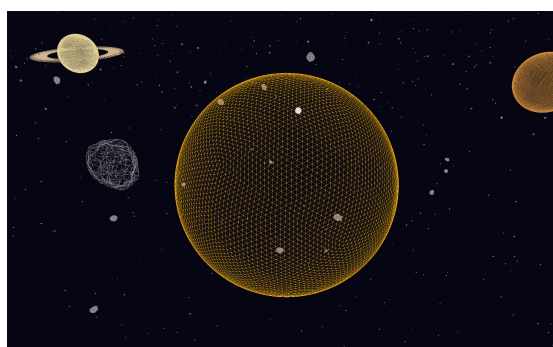


Figura 6: Resultado 4 — câmara orbital, aproximada com *wireframe* ativo.



Figura 7: Resultado 5 — câmara livre, anéis de Saturno.

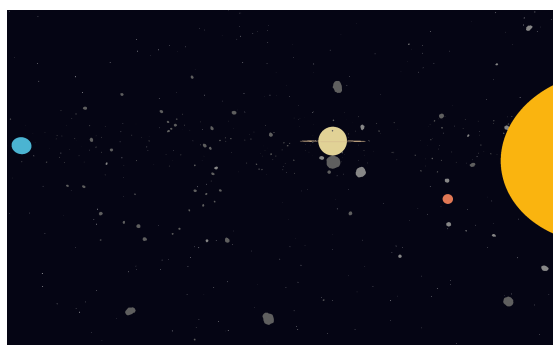


Figura 8: Resultado 6 — câmara livre, cinturão de asteroides.



Figura 9: Resultado 7 — câmara livre, luas de Júpiter.



Figura 10: Resultado 8 — câmara livre, Plutão e cinturão de *Kuiper*.

5. Conclusão

A segunda fase do trabalho prático foi concluída com sucesso. Foram implementadas as transformações geométricas hierárquicas pedidas — translação, rotação e escala — organizadas sob a forma de uma árvore de grupos lida a partir de um ficheiro XML. A *engine* passou a suportar uma renderização recursiva com pilha de matrizes, garantindo o correto isolamento das transformações entre grupos pai, filhos e irmãos.

Do lado do *generator*, foram acrescentadas três novas primitivas que vieram a revelar-se interessantes para a construção do modelo do sistema solar: o anel, o octaedro e o mecanismo de dispersão de instâncias (*scatter*). Este último, em particular, mostrou-se uma ferramenta versátil que vai além do pedido pelo enunciado, funcionando como um meta-gerador capaz de popular qualquer volume geométrico com instâncias aleatórias de qualquer modelo. O resultado foi aplicado com sucesso na geração dos cinturões de asteroides, do fundo estrelado, dos detritos e dos cometas, todos produzidos com o mesmo mecanismo e carregados pela *engine* de forma completamente transparente.

O modelo do sistema solar produzido incorpora os oito planetas, o Sol, a Lua, os anéis de Saturno, dois cinturões de asteroides, estrelas, detritos e cometas, demonstrando a expressividade da estrutura hierárquica de grupos implementada. A câmara livre adicionada permite explorar a cena de forma intuitiva, navegando livremente pelo espaço como se se estivesse dentro dele.

Estamos satisfeitos com o trabalho realizado e com a qualidade do resultado visual obtido, tendo ido consideravelmente além do mínimo pedido no enunciado.

Dadas algumas limitações da fase atual, temos ideias de desenvolvimento futuro que consideramos relevantes:

- VBOs (*Vertex Buffer Objects*): A fase atual utiliza `glBegin/glEnd` para a renderização, o que é ineficiente para cenas com muitos triângulos como os cinturões gerados pelo *scatter*. Isso já é visível no tempo de renderização da cena atual. A migração para VBOs permitirá melhorar a performance da cena.
- Curvas de Bézier e superfícies paramétricas: A próxima fase introduz curvas e superfícies. Uma aplicação diretamente relevante para o que desenvolvemos seria a geração de asteroides feitas por nós definida por superfícies de Bézier, em substituição do modelo OBJ externo atualmente utilizado. Tratar-se-ia de um asteroide totalmente gerado pelo nosso *generator*, sem dependência de recursos externos, e potencialmente mais leve e controlável em termos de detalhe.
- Órbitas animadas com curvas: As curvas de Bézier podem também ser usadas para definir as trajetórias orbitais dos planetas. Em vez de posições estáticas lidas do XML,

cada planeta seguiria uma curva paramétrica em função do tempo, produzindo um sistema solar animado e fisicamente mais plausível.

- *Logarithmic Depth Buffer*: O rácio *far/near* atual, necessário para acomodar simultaneamente asteroides pequenos e a esfera de estrelas distante, compromete a precisão do *z-buffer* e pode causar *z-fighting*. A implementação de um *logarithmic depth buffer* via *shader*, a introduzir quando os *shaders* forem suportados, distribuiria a precisão de forma uniforme em escala logarítmica, eliminando este problema de forma elegante.

Bibliografia

- [1] Khronos Group, «OpenGL - The Industry Standard for High Performance Graphics». [Em linha]. Disponível em: <https://www.opengl.org/>
- [2] H. S. M. Coxeter, *Regular Polytopes*, 3rd ed. Dover Publications, 1973.
- [3] L. Heninger, «TinyXML-2 — A Simple, Small, Efficient C++ XML Parser». 2023.
- [4] E. Angel, *Interactive Computer Graphics: A Top-Down Approach with OpenGL*, 4th ed. Addison-Wesley, 2005.
- [5] FreeGLUT Project, «FreeGLUT — The Free OpenGL Utility Toolkit». 2023.
- [6] Everios96, «8 Low Poly Asteroids». [Em linha]. Disponível em: <https://sketchfab.com/3d-models/8-low-poly-asteroids-6019fc70e28f4b5283f66a4d1d21741b>

Lista de Siglas e Acrónimos

API Application Programming Interface

CCW Counter-Clockwise

FOV Field of View

VBO Vertex Buffer Objects

XML Extensible Markup Language

Anexos

Anexo 1: Logo da Universidade do Minho

